Automatic IDs, None Defaults, and Refreshing Data

In the previous chapter, we saw how to add rows to the database using **SQLModel**.

Now let's talk a bit about why the `id` field **can't be `NULL`** on the database because it's a **primary key**, and we declare it using `Field(primary_key=True)`.

But the same `id` field actually **can be `None`** in the Python code, so we declare the type with `int | None`, and set the default value to `Field(default=None)`:

Python 3.10+

```python
# Code above omitted 👆

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Next, I'll show you a bit more about the synchronization of data between the database and the Python code.

When do we get an actual `int` from the database in that `id` field? Let's see all that. 👇

Create a New `Hero` Instance

When we create a new `Hero` instance, we don't set the `id`:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)
```

```
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

How `int | None` Helps

Because we don't set the `id`, it takes the Python's default value of `None` that we set in `Field(default=None)`.

This is the only reason why we define it with `int | None` and with a default value of `None`.

Because at this point in the code, **before interacting with the database**, the Python value could actually be `None`.

If we assumed that the `id` was *always* an `int` and added the type annotation without `int | None`, we could end up writing broken code, like:

```
next_hero_id = hero_1.id + 1
```

If we ran this code before saving the hero to the database and the `hero_1.id` was still `None`, we would get an error like:

```
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

But by declaring it with `int | None`, the editor will help us to avoid writing broken code by showing us a warning telling us that the code could be invalid if `hero_1.id` is `None`. 🔍

Print the Default `id` Values

We can confirm that by printing our heroes before adding them to the database:

Python 3.10+

```
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

That will output:

```bash
```

Notice they all have `id=None`.

That's the default value we defined in the `Hero` model class.

What happens when we `add` these objects to the **session**?

Add the Objects to the Session

After we add the `Hero` instance objects to the **session**, the IDs are *still* `None`.

We can verify by creating a session using a `with` block and adding the objects. And then printing them again:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
```

```
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

This will, again, output the `id` s of the objects as `None` :

```bash
```

As we saw before, the **session** is smart and doesn't talk to the database every time we prepare something to be changed, only after we are ready and tell it to `commit` the changes it goes and sends all the SQL to the database to store the data.

Commit the Changes to the Database

Then we can `commit` the changes in the session, and print again:

Python 3.10+

```python
# Code above omitted 👆

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

And now, something unexpected happens, look at the output, it seems as if the `Hero` instance objects had no data at all:

```bash
```

What happens is that SQLModel (actually SQLAlchemy) is internally marking those objects as "expired", they **don't have the latest version of their data**. This is because we could have some fields updated in the database, for example, imagine a field `updated_at: datetime` that was automatically updated when we saved changes.

The same way, other values could have changed, so the option the **session** has to be sure and safe is to just internally mark the objects as expired.

And then, next time we access each attribute, for example with:

```python
current_hero_name = hero_1.name
```

...SQLModel (actually SQLAlchemy) will make sure to contact the database and **get the most recent version of the data**, updating that field `name` in our object and then making it available for the rest of the Python expression. In the example above, at that point, Python would be able to continue executing and use that `hero_1.name` value (just updated) to put it in the variable `current_hero_name`.

All this happens automatically and behind the scenes. ✨

And here's the funny and strange thing with our example:

```
print("Hero 1:", hero_1)
```

We didn't access the object's attributes, like `hero.name`. We only accessed the entire object and printed it, so **SQLAlchemy has no way of knowing** that we want to access this object's data.

Print a Single Field

To confirm and understand how this **automatic expiration and refresh** of data when accessing attributes work, we can print some individual fields (instance attributes):

Python 3.10+

```
# Code above omitted 👆

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Now we are actually accessing the attributes, because instead of printing the whole object `hero_1`:

```python
print("Hero 1:", hero_1)
```

...we are now printing the `id` attribute in `hero.id`:

```python
print("Hero 1 ID:", hero_1.id)
```

By accessing the attribute, that **triggers** a lot of work done by SQLModel (actually SQLAlchemy) underneath to **refresh the data from the database**, set it in the object's `id` attribute, and make it available for the Python expression (in this case just to print it).

Let's see how it works:

```bash
```

Refresh Objects Explicitly

You just learnt how the **session** refreshes the data automatically behind the scenes, as a side effect, when you access an attribute.

But what if you want to **explicitly refresh** the data?

You can do that too with `session.refresh(object)`:

Python 3.10+

```
# Code above omitted 👆

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
```

```
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

When Python executes this code:

```python
session.refresh(hero_1)
```

...the **session** goes and makes the **engine** communicate with the database to get the recent data for this object `hero_1`, and then the **session** puts the data in the `hero_1` object and marks it as "fresh" or "not expired".

Here's how the output would look like:

```bash
```

This could be useful, for example, if you are building a web API to create heroes. And once a hero is created with some data, you return it to the client.

You wouldn't want to return an object that looks empty because the automatic magic to refresh the data was not triggered.

In this case, after committing the object to the database with the **session**, you could refresh it, and then return it to the client. This would ensure that the object has its fresh data.

Print Data After Closing the Session

Now, as a final experiment, we can also print data after the **session** is closed.

There are no surprises here, it still works:

Python 3.10+

```
# Code above omitted 👆

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    print("Before interacting with the database")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        print("After adding to the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        session.commit()

        print("After committing the session")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

        print("After committing the session, show IDs")
        print("Hero 1 ID:", hero_1.id)
        print("Hero 2 ID:", hero_2.id)
        print("Hero 3 ID:", hero_3.id)

        print("After committing the session, show names")
        print("Hero 1 name:", hero_1.name)
        print("Hero 2 name:", hero_2.name)
        print("Hero 3 name:", hero_3.name)

        session.refresh(hero_1)
        session.refresh(hero_2)
        session.refresh(hero_3)

        print("After refreshing the heroes")
        print("Hero 1:", hero_1)
        print("Hero 2:", hero_2)
        print("Hero 3:", hero_3)

    print("After the session closes")
    print("Hero 1:", hero_1)
    print("Hero 2:", hero_2)
    print("Hero 3:", hero_3)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

And the output shows again the same data:

```bash
```

Review All the Code

Now let's review all this code once again.

Tip

Each one of the numbered bubbles shows what each line will print in the output.

And as we created the **engine** with `echo=True`, we can see the SQL statements being executed at each step.

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"
```

```python
engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")  1
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")  2
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)  3

    print("Before interacting with the database")  4
    print("Hero 1:", hero_1)  5
    print("Hero 2:", hero_2)  6
    print("Hero 3:", hero_3)  7

    with Session(engine) as session:  8
        session.add(hero_1)  9
        session.add(hero_2)  10
        session.add(hero_3)  11

        print("After adding to the session")  12
        print("Hero 1:", hero_1)  13
        print("Hero 2:", hero_2)  14
        print("Hero 3:", hero_3)  15

        session.commit()  16

        print("After committing the session")  17
        print("Hero 1:", hero_1)  18
        print("Hero 2:", hero_2)  19
        print("Hero 3:", hero_3)  20

        print("After committing the session, show IDs")  21
        print("Hero 1 ID:", hero_1.id)  22
        print("Hero 2 ID:", hero_2.id)  23
        print("Hero 3 ID:", hero_3.id)  24

        print("After committing the session, show names")  25
        print("Hero 1 name:", hero_1.name)  26
        print("Hero 2 name:", hero_2.name)  27
        print("Hero 3 name:", hero_3.name)  28

        session.refresh(hero_1)  29
        session.refresh(hero_2)  30
        session.refresh(hero_3)  31

        print("After refreshing the heroes")  32
        print("Hero 1:", hero_1)  33
        print("Hero 2:", hero_2)  34
        print("Hero 3:", hero_3)  35
    36

    print("After the session closes")  37
    print("Hero 1:", hero_1)  38
    print("Hero 2:", hero_2)  39
    print("Hero 3:", hero_3)  40


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

1. Create the `hero_1`.

   Doesn't generate any output.

2. Create the `hero_2`.

   Doesn't generate any output.

3. Create the `hero_3`.

   Doesn't generate any output.

4. Print the line `"Before interacting with the database"`.

   Generates the output:

```
Before interacting with the database
```

5. Print the `hero_1` before interacting with the database.

   Generates the output:

```
Hero 1: id=None name='Deadpond' secret_name='Dive Wilson' age=None
```

6. Print the `hero_2` before interacting with the database.

   Generates the output:

```
Hero 2: id=None name='Spider-Boy' secret_name='Pedro Parqueador' age=None
```

7. Print the `hero_3` before interacting with the database.

   Generates the output:

```
Hero 3: id=None name='Rusty-Man' secret_name='Tommy Sharp' age=48
```

8. Create the `Session` in a `with` block.

   Doesn't generate any output.

9. Add the `hero_1` to the session.

   This still doesn't save it to the database.

   Doesn't generate any output.

10. Add the `hero_2` to the session.

    This still doesn't save it to the database.

    Doesn't generate any output.

11. Add the `hero_3` to the session.

    This still doesn't save it to the database.

    Doesn't generate any output.

12. Print the line `"After adding to the session"`.

    Generates the output:

```
After adding to the session
```

13. Print the `hero_1` after adding it to the session.

    It still has the same data as there hasn't been any interaction with the database yet. Notice that the `id` is still `None`.

    Generates the output:

```
Hero 1: id=None name='Deadpond' secret_name='Dive Wilson' age=None
```

14 Print the `hero_2` after adding it to the session.

It still has the same data as there hasn't been any interaction with the database yet. Notice that the `id` is still `None`.

Generates the output:

```
Hero 2: id=None name='Spider-Boy' secret_name='Pedro Parqueador' age=None
```

15 Print the `hero_3` after adding it to the session.

It still has the same data as there hasn't been any interaction with the database yet. Notice that the `id` is still `None`.

Generates the output:

```
Hero 3: id=None name='Rusty-Man' secret_name='Tommy Sharp' age=48
```

16 `commit` the **session**.

This will **save** all the data to the database. The **session** will use the **engine** to run a lot of SQL.

Generates the output:

```
INFO Engine BEGIN (implicit)
INFO Engine INSERT INTO hero (name, secret_name, age) VALUES (?, ?, ?)
INFO Engine [generated in 0.00018s] ('Deadpond', 'Dive Wilson', None)
INFO Engine INSERT INTO hero (name, secret_name, age) VALUES (?, ?, ?)
INFO Engine [cached since 0.0008968s ago] ('Spider-Boy', 'Pedro Parqueador', None)
INFO Engine INSERT INTO hero (name, secret_name, age) VALUES (?, ?, ?)
INFO Engine [cached since 0.001143s ago] ('Rusty-Man', 'Tommy Sharp', 48)
INFO Engine COMMIT
```

17 Print the line `"After committing the session"`.

Generates the output:

```
After committing the session
```

18 Print the `hero_1` after committing the session.

The `hero_1` is now internally marked as expired, and until it is refreshed, it looks like if it didn't contain any data.

Generates the output:

```
Hero 1:
```

19 Print the `hero_2` after committing the session.

The `hero_2` is now internally marked as expired, and until it is refreshed, it looks like if it didn't contain any data.

Generates the output:

```
Hero 2:
```

20 Print the `hero_3` after committing the session.

The `hero_3` is now internally marked as expired, and until it is refreshed, it looks like if it didn't contain any data.

Generates the output:

```
Hero 3:
```

21 Print the line `"After committing the session, show IDs"`.

Generates the output:

```
After committing the session, show IDs
```

22 Print the `hero_1.id`. A lot happens here.

Because we are accessing the attribute `id` of `hero_1`, **SQLModel** (actually SQLAlchemy) can detect that we are trying to access data from the `hero_1`.

It then detects that `hero_1` is currently associated with a **session** (because we added it to the session and committed it), and it is marked as expired.

Then with the **session**, it uses the **engine** to execute all the SQL to fetch the data for this object from the database.

Next it updates the object with the new data and marks it internally as "fresh" or "not expired".

Finally, it makes the ID value available for the rest of the Python expression. In this case, the Python expression just prints the ID.

Generates the output:

```
INFO Engine BEGIN (implicit)
INFO Engine SELECT hero.id AS hero_id, hero.name AS hero_name, hero.secret_name AS
hero_secret_name, hero.age AS hero_age
FROM hero
WHERE hero.id = ?
INFO Engine [generated in 0.00017s] (1,)

Hero 1 ID: 1
```

23 Print the `hero_2.id`.

A lot happens here, all the same stuff that happened at point 22, but for this `hero_2` object.

Generates the output:

```
INFO Engine SELECT hero.id AS hero_id, hero.name AS hero_name, hero.secret_name AS
hero_secret_name, hero.age AS hero_age
FROM hero
WHERE hero.id = ?
INFO Engine [cached since 0.001245s ago] (2,)

Hero 2 ID: 2
```

24 Print the `hero_3.id`.

A lot happens here, all the same stuff that happened at point 22, but for this `hero_3` object.

Generates the output:

```
INFO Engine SELECT hero.id AS hero_id, hero.name AS hero_name, hero.secret_name AS
hero_secret_name, hero.age AS hero_age
FROM hero
WHERE hero.id = ?
INFO Engine [cached since 0.002215s ago] (3,)


Hero 3 ID: 3
```

25 Print the line `"After committing the session, show names"`.

Generates the output:

```
After committing the session, show names
```

26 Print the `hero_1.name`.

Because `hero_1` is still fresh, no additional data is fetched, no additional SQL is executed, and the name is available.

Generates the output:

```
Hero 1 name: Deadpond
```

27 Print the `hero_2.name`.

Because `hero_2` is still fresh, no additional data is fetched, no additional SQL is executed, and the name is available.

Generates the output:

```
Hero 2 name: Spider-Boy
```

28 Print the `hero_3.name`.

Because `hero_3` is still fresh, no additional data is fetched, no additional SQL is executed, and the name is available.

Generates the output:

```
Hero 3 name: Rusty-Man
```

29 Explicitly refresh the `hero_1` object.

The **session** will use the **engine** to execute the SQL necessary to fetch fresh data from the database for the `hero_1` object.

Generates the output:

```
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.id = ?
INFO Engine [generated in 0.00024s] (1,)
```

30 Explicitly refresh the `hero_2` object.

The **session** will use the **engine** to execute the SQL necessary to fetch fresh data from the database for the `hero_2` object.

Generates the output:

```
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.id = ?
INFO Engine [cached since 0.001487s ago] (2,)
```

31 Explicitly refresh the `hero_3` object.

The **session** will use the **engine** to execute the SQL necessary to fetch fresh data from the database for the `hero_3` object.

Generates the output:

```
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.id = ?
INFO Engine [cached since 0.002377s ago] (3,)
```

32 Print the line `"After refreshing the heroes"`.

Generates the output:

```
After refreshing the heroes
```

33 Print the `hero_1`.

> **Info**
>
> Even if the `hero_1` wasn't fresh, this would **not** trigger a `refresh` making the **session** use the **engine** to fetch data from the database because it is not accessing an attribute.

Because the `hero_1` is fresh it has all it's data available.

Generates the output:

```
Hero 1: age=None id=1 name='Deadpond' secret_name='Dive Wilson'
```

34 Print the `hero_2`.

> **Info**
>
> Even if the `hero_2` wasn't fresh, this would **not** trigger a `refresh` making the **session** use the **engine** to fetch data from the database because it is not accessing an attribute.

Because the `hero_2` is fresh it has all it's data available.

Generates the output:

```
Hero 2: age=None id=2 name='Spider-Boy' secret_name='Pedro Parqueador'
```

35 Print the `hero_3`.

> **Info**
>
> Even if the `hero_3` wasn't fresh, this would **not** trigger a `refresh` making the **session** use the **engine** to fetch data from the database because it is not accessing an attribute.

Because the `hero_3` is fresh it has all it's data available.

Generates the output:

```
Hero 3: age=48 id=3 name='Rusty-Man' secret_name='Tommy Sharp'
```

36 The `with` block ends here (there's no more indented code), so the **session** is closed, running all it's closing code.

This includes doing a `ROLLBACK` of any possible transaction that could have been started.

Generates the output:

```
INFO Engine ROLLBACK
```

37 Print the line `"After the session closes"`.

Generates the output:

```
After the session closes
```

38 Print the `hero_1` after closing the session.

Generates the output:

```
Hero 1: age=None id=1 name='Deadpond' secret_name='Dive Wilson'
```

39 Print the `hero_2` after closing the session.

Generates the output:

```
Hero 2: age=None id=2 name='Spider-Boy' secret_name='Pedro Parqueador'
```

40 Print the `hero_3` after closing the session.

Generates the output:

```
Hero 3: age=48 id=3 name='Rusty-Man' secret_name='Tommy Sharp'
```

And here's all the output generated by running this program, all together:

bash

Recap

You read all that! That was a lot! Have some cake, you earned it. 🍰

We discussed how the **session** uses the **engine** to send SQL to the database, to create data and to fetch data too. How it keeps track of "**expired**" and "**fresh**" data. At which moments it **fetches data automatically** (when accessing instance attributes) and how that data is synchronized between objects in memory and the database via the **session**.

If you understood all that, now you know a lot about **SQLModel**, SQLAlchemy, and how the interactions from Python with databases work in general.

If you didn't get all that, it's fine, you can always come back later to `refresh` the concepts.

I think this might be one of the main types of bugs that cause problems and makes you scratch your head. So, good job studying it! 💪